Assignment-17.3

HT NO:2303A51306

BATCH:05

Task-01:

Prompt: Write a Python program using the pandas library to clean an employee dataset (employee_data.csv). The program should handle missing values by filling the salary column with its median, replacing missing designation values with "unknown", and filling missing experience with the mean. It should remove duplicate records based on employee_id, standardize the designation column by converting text to lowercase and removing extra spaces, and then apply label encoding to the designation column. Additionally, perform one-hot encoding on the employment_type column. The program must display the dataset before and after cleaning and finally save the cleaned dataset as clean_employee_data.csv. Ensure the code is well-structured and includes clear comments.

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, OneHotEncoder

# --- 0. Load the Dataset ---
# Assuming 'employee_data.csv' exists in the current directory.
# If not, you would need to create it first or adjust the path.

df = pd.read_csv('employee_data.csv')

print("--- Original Dataset (First 5 Rows) ---")
display(df.head())
print(f"Original dataset shape: {df.shape}")
print("Original Missing Values:\n", df.isnull().sum())
print(f"Original number of unique employee_id: {df['employee_id'].nunique()}\n")

# Keep a copy of the original DataFrame for comparison after cleaning
df_original = df.copy()
```

```
--- Original Dataset (First 5 Rows) ---
```

1. Handle Missing Values

- **salary** : Fill with the median salary (robust to outliers).
- **designation** : Fill with the string "unknown".
- **experience** : Fill with the mean experience.

```python
# Calculate median salary and mean experience
median_salary = df['salary'].median()
mean_experience = df['experience'].mean()

# Fill missing values using fillna()
df['salary'] = df['salary'].fillna(median_salary)
df['designation'] = df['designation'].fillna('unknown')
df['experience'] = df['experience'].fillna(mean_experience)

print("--- Missing Values After Handling ---")
print(df.isnull().sum())
print(f"Median Salary Used: {median_salary}")
print(f"Mean Experience Used: {mean_experience}\n")
```

2. Remove Duplicate Records

- Remove rows with duplicate `employee_id`, keeping the first occurrence.

```
print(f"Shape before removing duplicates: {df.shape}")

# Remove duplicates based on 'employee_id', keeping the first record
df.drop_duplicates(subset=['employee_id'], keep='first', inplace=True)

print(f"Shape after removing duplicates: {df.shape}")
print(f"Number of unique employee_id after removing duplicates: {df['employee_id'].nunique()}\n")
```

3. Standardize Text

- Convert `designation` values to lowercase.
- Strip leading/trailing spaces from all text (object) columns.

```
# Convert 'designation' to lowercase
df['designation'] = df['designation'].str.lower()

# Strip spaces from all object (string) columns
for col in df.select_dtypes(include=['object']).columns:
    df[col] = df[col].str.strip()

print("--- Sample of Text Columns After Standardization ---")
display(df[['name', 'designation']].head())
print("\n")
```

```
--- Sample of Text Columns After Standardization ---
```

4. Encode Categorical Features

- **Label Encoding for `designation`** : Assign a unique integer to each category.
- **One-Hot Encoding for `employment_type`** : Create new binary columns for each category.

```
# --- Label Encoding for 'designation' ---
le = LabelEncoder()
df['designation_encoded'] = le.fit_transform(df['designation'])

print("--- Label Encoding Mapping (Designation) ---")
designation_mapping = dict(zip(le.classes_, le.transform(le.classes_)))
print(designation_mapping)

# --- One-Hot Encoding for 'employment_type' ---
# Initialize OneHotEncoder with handle_unknown='ignore' to prevent errors for unseen categori
# and sparse_output=False for a dense NumPy array.
# drop='first' avoids multicollinearity (dummy variable trap).

ohe = OneHotEncoder(handle_unknown='ignore', sparse_output=False, drop='first')

# Fit and transform the 'employment_type' column (requires reshaping for a single column)
ohe_features = ohe.fit_transform(df[['employment_type']])

# Get the feature names for the new one-hot encoded columns
ohe_feature_names = ohe.get_feature_names_out(['employment_type'])
```

5. Final Output: Display and Save Cleaned Dataset

- Display the first few rows of the final cleaned dataset.
- Save the cleaned dataset to `clean_employee_data.csv`.

```
print("\n--- Final Cleaned Dataset (First 5 Rows) ---")
display(df.head())
print(f"Cleaned dataset shape: {df.shape}")

print("\n--- Final Missing Values Check (Should be all zeros) ---")
print(df.isnull().sum())

# Save the cleaned dataset to a new CSV file
df.to_csv('clean_employee_data.csv', index=False)

print("\nCleaned dataset successfully saved as 'clean_employee_data.csv'\n")
```

Output:

```
--- Final Cleaned Dataset (First 5 Rows) ---
```

	employee_id	name	salary	designation	experience	employment_type	designation_encoded	employment_type_Full-time	employment_type_Part-time
0	101	Alice Smith	70000.0	software engineer	5.0	Full-time	6	1.0	0.0
1	102	Bob Johnson	85000.0	project manager	8.0	Full-time	5	1.0	0.0
2	103	Charlie Brown	75000.0	data scientist	3.0	Contract	1	0.0	0.0
3	104	David Lee	60000.0	software engineer	5.0	Full-time	6	1.0	0.0
4	105	Eve Davis	92000.0	unknown	10.0	Part-time	8	0.0	1.0

```
Cleaned dataset shape: (15, 9)

--- Final Missing Values Check (Should be all zeros) ---
employee_id                 0
name                        0
salary                      0
designation                 0
experience                  0
employment_type             0
designation_encoded         0
employment_type_Full-time   0
employment_type_Part-time   0
dtype: int64

Cleaned dataset successfully saved as 'clean_employee_data.csv'
```

Explanation:

The program loads an employee dataset using pandas and displays it to show the original data before any preprocessing. It then handles missing values by filling the salary column with the median, replacing missing designation entries with "unknown", and updating missing experience values with the mean to ensure smooth analysis. Duplicate entries based on employee_id are removed so that each employee record remains unique.

Next, the program standardizes text data by removing extra spaces and converting job titles to lowercase for uniformity. It then transforms categorical columns into numerical formats suitable for machine learning models. Finally, the cleaned dataset is saved as a new CSV file for further use.

Task-02:

Prompt: Use the dataset file student_data.csv to perform preprocessing by first importing the necessary libraries such as pandas, numpy, and sklearn. Convert the exam_date column into datetime format and derive additional columns like month, semester, and day from it. Apply Min-Max scaling to normalize the total_marks column, and identify as well as handle outliers using the IQR method. Display the dataset both before and after preprocessing to observe the changes, and finally save the processed data as processed_student_data.csv with an option to download it. Ensure the code is neatly structured, well-commented, and ready to execute**.**

Code:

Student Performance Data Preprocessing Program

This program demonstrates key preprocessing steps for a student performance dataset using `pandas` and `numpy`, along with `sklearn` for normalization.

First, we'll create a sample `student_data.csv` file. In a real scenario, you would replace this step with loading your actual data.

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# --- 0. Create Sample student_data.csv (if it doesn't exist) ---
# This step is for demonstration. In a real scenario, you'd load your existing file.
try:
    df = pd.read_csv('student_data.csv')
    print("student_data.csv found. Loading existing data.")
except FileNotFoundError:
    print("student_data.csv not found. Creating sample data.")
    data = {
        'student_id': range(1, 21),
        'exam_date': pd.to_datetime([
            '2023-01-15', '2023-02-20', '2023-03-10', '2023-04-05', '2023-05-18',
            '2023-06-25', '2023-07-01', '2023-08-12', '2023-09-01', '2023-10-10',
            '2023-11-22', '2023-12-01', '2024-01-08', '2024-02-14', '2024-03-20',
            '2023-01-20', '2023-05-01', '2023-09-15', '2023-04-10', '2024-02-29' # Leap year date
        ]),
        'math_score': [75, 80, 60, 90, 70, 85, 95, 65, 78, 88, 72, 91, 55, 82, 68, 70, 93, 79, 81, 76],
        'science_score': [80, 75, 70, 85, 65, 90, 88, 72, 82, 70, 78, 92, 60, 85, 75, 78, 90, 80, 77, 81],
```


```python
        '2023-01-20', '2023-05-01', '2023-09-15', '2023-04-10', '2024-02-29' # Leap year date
    ]),
    'math_score': [75, 80, 60, 90, 70, 85, 95, 65, 78, 88, 72, 91, 55, 82, 68, 70, 93, 79, 81, 76],
    'science_score': [80, 75, 70, 85, 65, 90, 88, 72, 82, 70, 78, 92, 60, 85, 75, 78, 90, 80, 77, 81],
```


```python
    'history_score': [70, 85, 55, 92, 75, 80, 90, 68, 85, 75, 80, 88, 62, 80, 70, 72, 95, 82, 80, 78],
    'total_marks': [
        225, 240, 185, 267, 210, 255, 273, 205, 245, 233, 230, 271, 177, 247, 213,
        220, 278, 241, 238, 235
    ]
}

# Introduce some outliers for total_marks
data['total_marks'][2] = 50 # Low outlier
data['total_marks'][6] = 350 # High outlier
data['total_marks'][12] = 40 # Low outlier

df = pd.DataFrame(data)
df.to_csv('student_data.csv', index=False)
print("Sample 'student_data.csv' created and loaded successfully!")

# --- 1. Load the Dataset and Display Initial State ---
print("\n--- Original Dataset (First 5 Rows) ---")
display(df.head())
print(f"Original dataset shape: {df.shape}")
print("Original Data Info:")
df.info()
print("\nOriginal Total Marks Statistics:\n", df['total_marks'].describe())

df_original = df.copy() # Keep a copy for comparison
```

2. Convert `exam_date` to Datetime and Extract New Features

We will convert the `exam_date` column to a proper datetime format if it's not already, and then extract the month, day, and a custom 'semester' feature from it. We'll define semesters as:

- **Semester 1**: Months 1-6 (January to June)
- **Semester 2**: Months 7-12 (July to December)

```
# Convert 'exam_date' to datetime format
df['exam_date'] = pd.to_datetime(df['exam_date'])

# Extract new features
df['exam_month'] = df['exam_date'].dt.month
df['exam_day'] = df['exam_date'].dt.day

# Extract 'semester' (custom logic: Jan-Jun is S1, Jul-Dec is S2)
df['semester'] = df['exam_month'].apply(lambda x: 1 if 1 <= x <= 6 else 2)

print("--- Dataset After Datetime Conversion and Feature Extraction (First 5 Rows) ---")
display(df.head())
```

3. Normalize `total_marks` using Min-Max Normalization

Min-Max normalization scales features to a fixed range, typically 0 to 1. This is useful for algorithms that are sensitive to the magnitude of values, such as neural networks.

The formula for Min-Max normalization is: $X_{normalized} = (X - X_{min})/(X_{max} - X_{min})$

```
# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Reshape the 'total_marks' column for the scaler and apply transformation
df['total_marks_normalized'] = scaler.fit_transform(df[['total_marks']])

print("--- Total Marks after Min-Max Normalization (First 5 Rows) ---")
display(df[['total_marks', 'total_marks_normalized']].head())
print("\nNormalized Total Marks Statistics:\n", df['total_marks_normalized'].describe())
```

4. Detect and Handle Outliers in `total_marks` using IQR Method

Outliers can skew statistical analyses and model training. The Interquartile Range (IQR) method is a robust way to id
falling outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$ are considered outliers.

For handling, we'll implement **clamping**, where outliers are replaced with the respective upper or lower bound.

```
# Calculate Q1 (25th percentile) and Q3 (75th percentile) for 'total_marks'
Q1 = df['total_marks'].quantile(0.25)
Q3 = df['total_marks'].quantile(0.75)
IQR = Q3 - Q1

# Define outlier bounds
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Identify outliers
outliers_mask = (df['total_marks'] < lower_bound) | (df['total_marks'] > upper_bound)
num_outliers = outliers_mask.sum()

print(f"\n--- Outlier Detection for 'total_marks' ---")
print(f"Q1: {Q1:.2f}")
print(f"Q3: {Q3:.2f}")
print(f"IQR: {IQR:.2f}")
print(f"Lower Bound: {lower_bound:.2f}")
print(f"Upper Bound: {upper_bound:.2f}")
print(f"Number of outliers detected: {num_outliers}")
```

5. Display Dataset Before and After Preprocessing & Save Transformed Dataset

Finally, we'll display a comparison of the original and processed data, and save the cleaned and transformed DataFrame to a new CSV file.

```
print("\n--- Original Dataset (First 5 Rows for comparison) ---")
display(df_original.head())

print("\n--- Final Preprocessed Dataset (First 5 Rows) ---")
display(df.head())
print(f"Processed dataset shape: {df.shape}")

# You might choose which 'total_marks' column to keep (original, normalized, or handled)
# For the final output, let's keep the handled and normalized version and drop the original 'total_marks' if not needed.
# For this example, we will keep all for clarity, but in a real scenario, you would select columns.

# Save the transformed dataset to a new CSV file
df.to_csv('processed_student_data.csv', index=False)

print("\nProcessed dataset successfully saved as 'processed_student_data.csv'\n")
```

Output:

```
--- Original Dataset (First 5 Rows for comparison) ---
```

	student_id	exam_date	math_score	science_score	history_score	total_marks
0	1	2023-01-15	75	80	70	225
1	2	2023-02-20	80	75	85	240
2	3	2023-03-10	60	70	55	50
3	4	2023-04-05	90	85	92	267
4	5	2023-05-18	70	65	75	210

```
--- Final Preprocessed Dataset (First 5 Rows) ---
```

	student_id	exam_date	math_score	science_score	history_score	total_marks	exam_month	exam_day	semester	total_marks_normalized	total_marks_handled
0	1	2023-01-15	75	80	70	225	1	15	1	0.596774	225.000
1	2	2023-02-20	80	75	85	240	2	20	1	0.645161	240.000
2	3	2023-03-10	60	70	55	50	3	10	1	0.032258	172.125
3	4	2023-04-05	90	85	92	267	4	5	1	0.732258	267.000
4	5	2023-05-18	70	65	75	210	5	18	1	0.548387	210.000

```
Processed dataset shape: (20, 11)

Processed dataset successfully saved as 'processed_student_data.csv'
```

Explanation:

The program begins by importing essential libraries such as pandas, numpy, and sklearn, and then loads the student dataset into Google Colab. It transforms the exam_date column into datetime format to enable accurate date-related operations. From this column, additional features like month, semester, and day are derived to enhance the dataset for analysis and predictive tasks.

Next, the total_marks column is scaled using Min-Max normalization to bring all values within a uniform range. The code also identifies outliers in the marks using the IQR method and handles them appropriately by either removing or adjusting extreme values. In the end, the processed dataset is stored as a new CSV file for future use.

Task-03:

Prompt:

Create a complete Python program to preprocess a student health dataset (student_health.csv) by first importing the required libraries such as pandas and components from sklearn, and then loading the dataset. Handle missing values by filling the height and weight columns with their mean values, and the BMI column with the median. Convert categorical features like medical_condition and fitness_status into numerical form using label encoding.

Next, standardize the numerical columns (height, weight, BMI) using StandardScaler to bring them to a common scale. Split the dataset into training and testing sets with an 80:20 ratio. Display a few samples of the processed data to verify the changes, and finally save the cleaned and transformed dataset as processed_student_health.csv.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
# --- 0. Create Sample student_health.csv (if it doesn't exist) ---
# This step is for demonstration. In a real scenario, you'd load your existing file.
try:
    df = pd.read_csv('student_health.csv')
    print("student_health.csv found. Loading existing data.")
except FileNotFoundError:
    print("student_health.csv not found. Creating sample data.")
    data = {
        'student_id': range(1, 21),
        'age': [15, 16, 15, 17, 16, 15, 17, 16, 15, 17, 16, 15, 17, 16, 15, 17, 16, 15, 17, 16],
        'gender': ['Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female',
                   'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female', 'Male', 'Female'],
        'height_cm': [165, 158, 170, 163, 175, 160, 168, np.nan, 172, 161, 166, 159, 173, 164, 169, 162, 171, 157, 174, np.nan],
        'weight_kg': [55, 50, 68, 53, 65, np.nan, 58, 52, 62, 51, 56, 49, 63, 54, 59, 52, np.nan, 48, 64, 57],
        'bmi': [20.2, 20.0, 20.8, 19.9, 21.2, 20.3, 20.5, 20.1, 21.0, 19.7, 20.4, np.nan, 20.7, 20.0, 20.6, 19.8, 20.9, 19.6, 21.1, 20.2],
        'medical_condition': ['None', 'Asthma', 'None', 'Allergy', 'None', 'Asthma', 'None', 'None', 'Allergy', 'None',
                              'None', 'Asthma', 'None', 'None', 'Allergy', 'None', 'Asthma', 'None', 'None', 'None'],
        'fitness_status': ['Good', 'Average', 'Good', 'Average', 'Excellent', 'Average', 'Good', 'Good', 'Excellent', 'Average',
                           'Good', 'Average', 'Excellent', 'Good', 'Average', 'Good', 'Average', 'Good', 'Excellent', 'Average']
    }

    df = pd.DataFrame(data)
    df.to_csv('student_health.csv', index=False)
    print("Sample 'student_health.csv' created and loaded successfully!")
# --- 1. Load the Dataset ---
print("\n--- Original Dataset (First 5 Rows) ---")
display(df.head())
print(f"Original dataset shape: {df.shape}")
print("Original Missing Values:\n", df.isnull().sum())
df_original = df.copy() # Keep a copy for comparison
```

2. Handle Missing Values

- `height_cm` and `weight_kg` : Fill with the mean value.
- `bmi` : Fill with the median value (more robust to outliers).

```python
# Calculate mean for height and weight
mean_height = df['height_cm'].mean()
mean_weight = df['weight_kg'].mean()

# Calculate median for BMI
median_bmi = df['bmi'].median()

# Fill missing values
df['height_cm'].fillna(mean_height, inplace=True)
df['weight_kg'].fillna(mean_weight, inplace=True)
df['bmi'].fillna(median_bmi, inplace=True)

print("--- Missing Values After Handling ---")
print(df.isnull().sum())
print(f"Mean Height Used: {mean_height:.2f}")
print(f"Mean Weight Used: {mean_weight:.2f}")
print(f"Median BMI Used: {median_bmi:.2f}\n")
```

```
--- Missing Values After Handling ---
student_id          0
age                 0
gender              0
height_cm           0
weight_kg           0
```

3. Encode Categorical Columns

- **medical_condition** and **fitness_status** : Apply Label Encoding to convert categories into numerical representations.

```
le = LabelEncoder()

# Apply Label Encoding to 'medical_condition'
df['medical_condition_encoded'] = le.fit_transform(df['medical_condition'])
print("--- Label Encoding Mapping (Medical Condition) ---")
print(dict(zip(le.classes_, le.transform(le.classes_))))

# Apply Label Encoding to 'fitness_status'
df['fitness_status_encoded'] = le.fit_transform(df['fitness_status'])
print("\n--- Label Encoding Mapping (Fitness Status) ---")
print(dict(zip(le.classes_, le.transform(le.classes_))))

print("\n--- Sample with Encoded Categorical Features ---")
display(df[['medical_condition', 'medical_condition_encoded', 'fitness_status', 'fitness_status_encoded']].head())
```

```
--- Label Encoding Mapping (Medical Condition) ---
{'Allergy': np.int64(0), 'Asthma': np.int64(1), 'None': np.int64(2)}

--- Label Encoding Mapping (Fitness Status) ---
{'Average': np.int64(0), 'Excellent': np.int64(1), 'Good': np.int64(2)}

--- Sample with Encoded Categorical Features ---
```

	medical_condition	medical_condition_encoded	fitness_status	fitness_status_encoded
0	None	2	Good	2
1	Asthma	1	Average	0

4. Scale Numerical Columns

- **height_cm**, **weight_kg**, **bmi** : Apply `StandardScaler` to transform these features to have a mean of 0 and a standard deviation of 1. This is crucial for many machine learning algorithms.

```
scaler = StandardScaler()

# Define numerical columns to scale
numerical_cols = ['height_cm', 'weight_kg', 'bmi']

# Apply StandardScaler
df[numerical_cols] = scaler.fit_transform(df[numerical_cols])

print("--- Sample with Scaled Numerical Features (First 5 Rows) ---")
display(df[numerical_cols].head())
print("\nScaled Numerical Features Statistics:\n")
print(df[numerical_cols].describe())
```

```
--- Sample with Scaled Numerical Features (First 5 Rows) ---
```

	height_cm	weight_kg	bmi
0	-0.177075	-0.204124	-0.357284
1	-1.489511	-1.224745	-0.790356
2	0.760380	0.816497	0.941931
3	-0.552056	-0.612372	-1.006892
4	1.697834	1.837117	1.808075

5. Split Dataset into Training and Testing Sets

- Split the dataset into an 80% training set and a 20% testing set. This is a standard practice for evaluating machine learning models.

```python
# Define features (X) and target (y) - assuming 'student_id', original categorical columns are dropped for X
# For simplicity, let's say we want to predict 'fitness_status_encoded' from other features.

X = df.drop(columns=['student_id', 'medical_condition', 'fitness_status', 'fitness_status_encoded'])
y = df['fitness_status_encoded']

# Split the dataset (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y) # stratify for balanced classes

print(f"Shape of X_train: {X_train.shape}")
print(f"Shape of X_test: {X_test.shape}")
print(f"Shape of y_train: {y_train.shape}")
print(f"Shape of y_test: {y_test.shape}\n")
```

```
Shape of X_train: (16, 6)
Shape of X_test: (4, 6)
Shape of y_train: (16,)
Shape of y_test: (4,)
```

6. Display Processed Data Samples & Save Transformed Dataset

- Display the first few rows of the final processed dataset.
- Save the final processed DataFrame to `processed_student_health.csv`.

```python
print("--- Original Dataset (First 5 Rows for comparison) ---")
display(df_original.head())

print("\n--- Final Processed Dataset (First 5 Rows) ---")
display(df.head())
print(f"Processed dataset shape: {df.shape}")

# Save the transformed dataset to a new CSV file
df.to_csv('processed_student_health.csv', index=False)

print("\nProcessed dataset successfully saved as 'processed_student_health.csv'\n")
```

```
--- Original Dataset (First 5 Rows for comparison) ---
```

Output:

```
print("\nProcessed dataset successfully saved as 'processed_student_health.csv'\n")

--- Original Dataset (First 5 Rows for comparison) ---
```

	student_id	age	gender	height_cm	weight_kg	bmi	medical_condition	fitness_status
0	1	15	Male	165.0	55.0	20.2	None	Good
1	2	16	Female	158.0	50.0	20.0	Asthma	Average
2	3	15	Male	170.0	60.0	20.8	None	Good
3	4	17	Female	163.0	53.0	19.9	Allergy	Average
4	5	16	Male	175.0	65.0	21.2	None	Excellent

```
--- Final Processed Dataset (First 5 Rows) ---
```

	student_id	age	gender	height_cm	weight_kg	bmi	medical_condition	fitness_status	medical_condition_encoded	fitness_status_encoded
0	1	15	Male	-0.177075	-0.204124	-0.357284	None	Good	2	2
1	2	16	Female	-1.489511	-1.224745	-0.790356	Asthma	Average	1	0
2	3	15	Male	0.760380	0.816497	0.941931	None	Good	2	2
3	4	17	Female	-0.552056	-0.612372	-1.006892	Allergy	Average	0	0
4	5	16	Male	1.697834	1.837117	1.808075	None	Excellent	2	1

```
Processed dataset shape: (20, 10)

Processed dataset successfully saved as 'processed_student_health.csv'
```

Explanation:

The code starts by importing pandas for managing data and sklearn utilities for preprocessing and splitting the dataset. It then loads the student health dataset and handles missing values by filling height and weight with their mean values, while BMI is replaced with the median to minimize the effect of extreme values.

Next, categorical fields such as medical condition and fitness status are transformed into numerical form using label encoding, making them suitable for machine learning models. The numerical attributes are then standardized using StandardScaler, ensuring they have a mean of 0 and a standard deviation of 1 for balanced model performance. Finally, the dataset is split into training and testing sets for analysis, and the processed data is saved as a new CSV file.

Task-04:

Prompt: Develop a Python program using pandas and sklearn to clean a ridesharing dataset (rides_data.csv). Begin by loading the dataset, then standardize the pickup_location and drop_location columns by converting text to lowercase and removing extra spaces for consistency. Handle missing values in the fare_amount column by filling them with the median fare.

Next, eliminate duplicate records based on ride_id to ensure each trip is unique. Normalize the driver_rating column using Min-Max scaling so values fall within a consistent range. Generate a summary that reports the number of missing values handled, duplicates removed, and columns that were transformed. Display the dataset before and after cleaning, and finally save the processed dataset as clean_rides_data.csv.

Code:

```python
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# --- 0. Create Sample ride_data.csv (if it doesn't exist) ---
# This step is for demonstration. In a real scenario, you'd load your existing file.
try:
    df = pd.read_csv('ride_data.csv')
    print("ride_data.csv found. Loading existing data.")
except FileNotFoundError:
    print("ride_data.csv not found. Creating sample data.")
    data = {
        'ride_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19],
        'pickup_location': ['Downtown ', ' Airport ', 'Suburbs', 'Downtown', 'Airport ', 'University', 'Downtown', 'Airport', ' Suburbs', 'University', 'Downtown', 'Airport', 'Suburbs', 'Downtown
        'drop_location': ['Uptown', 'Downtown', 'Airport', 'University', 'Suburbs', 'Downtown', 'Uptown ', 'University', ' Airport', 'Downtown', 'Uptown', 'University', 'Airport', 'University',
        'fare_amount': [25.50, 45.00, np.nan, 20.00, 40.00, 15.00, 10.00, 50.00, 12.00, np.nan, 15.50, 42.00, 18.00, 35.00, 41.00, 16.00, 37.00, 46.00, 13.00, 17.00],
        'driver_rating': [4.5, 3.8, 4.9, 4.2, 3.5, 4.0, 4.7, 3.9, np.nan, 4.1, 4.5, 3.7, 4.6, 4.3, 4.6, 4.1, 4.4, 3.8, 4.9, 4.0],
        'ride_timestamp': pd.to_datetime([
            '2023-01-01 10:00:00', '2023-01-01 11:30:00', '2023-01-01 12:00:00', '2023-01-01 13:15:00', '2023-01-01 14:00:00',
            '2023-01-02 09:00:00', '2023-01-02 10:30:00', '2023-01-02 12:00:00', '2023-01-02 13:45:00', '2023-01-02 15:00:00',
            '2023-01-03 10:00:00', '2023-01-03 08:00:00', '2023-01-03 09:15:00', '2023-01-03 10:00:00', '2023-01-03 11:30:00',
            '2023-01-04 12:00:00', '2023-01-04 13:00:00', '2023-01-04 14:00:00', '2023-01-04 15:00:00', '2023-01-04 16:00:00'
        ])
    }

    df = pd.DataFrame(data)
    df.to_csv('ride_data.csv', index=False)
    print("Sample 'ride_data.csv' created and loaded successfully!")

# --- 1. Load the Dataset and Display Initial State ---
print("\n--- Original Dataset (First 5 Rows) ---")
display(df.head())
print("Original dataset shape: {df.shape}")
print("Original Missing Values:\n", df.isnull().sum())
```

2. Standardize `pickup_location` and `drop_location` Text

We will convert these text columns to lowercase and remove any leading or trailing whitespace to ensure consistency.

```python
# Standardize 'pickup_location' and 'drop_location'
df['pickup_location'] = df['pickup_location'].str.lower().str.strip()
df['drop_location'] = df['drop_location'].str.lower().str.strip()

print("--- Sample of Location Columns After Text Standardization ---")
display(df[['pickup_location', 'drop_location']].head())
print("\n")
```

```
--- Sample of Location Columns After Text Standardization ---
```

	pickup_location	drop_location
0	downtown	uptown
1	airport	downtown
2	suburbs	airport
3	downtown	university
4	airport	suburbs

3. Fill Missing `fare_amount` Values

Missing `fare_amount` values will be filled with the median fare to minimize the impact of potential outliers.

```
median_fare = df['fare_amount'].median()
initial_missing_fare = df['fare_amount'].isnull().sum()
df['fare_amount'].fillna(median_fare, inplace=True)
missing_fare_fixed = initial_missing_fare - df['fare_amount'].isnull().sum()

print(f"Median Fare Used to fill missing values: {median_fare:.2f}")
print(f"Number of missing 'fare_amount' values fixed: {missing_fare_fixed}")
print("\n--- Missing Values After Fare Amount Handling ---")
print(df.isnull().sum())
print("\n")
```

```
Median Fare Used to fill missing values: 25.50
Number of missing 'fare_amount' values fixed: 2

--- Missing Values After Fare Amount Handling ---
ride_id            0
pickup_location    0
drop_location      0
fare_amount        0
driver_rating      1
ride_timestamp     0
dtype: int64
```

4. Remove Duplicate Ride Entries

We will remove duplicate records based on `ride_id`, keeping only the first occurrence.

```
print(f"Shape before removing duplicates: {df.shape}")
initial_num_rows = df.shape[0]

# Remove duplicates based on 'ride_id', keeping the first record
df.drop_duplicates(subset=['ride_id'], keep='first', inplace=True)
duplicates_removed_count = initial_num_rows - df.shape[0]

print(f"Shape after removing duplicates: {df.shape}")
print(f"Number of duplicates removed: {duplicates_removed_count}")
print(f"Number of unique ride_id after removing duplicates: {df['ride_id'].nunique()}\n")
```

```
Shape before removing duplicates: (20, 6)
Shape after removing duplicates: (19, 6)
Number of duplicates removed: 1
Number of unique ride_id after removing duplicates: 19
```

5. Normalize `driver_rating`

We'll use Min-Max normalization to scale `driver_rating` to a range between 0 and 1. First, we need to fill any missi
using the mean as it's a numerical feature.

```
mean_driver_rating = df['driver_rating'].mean()
initial_missing_driver_rating = df['driver_rating'].isnull().sum()
df['driver_rating'].fillna(mean_driver_rating, inplace=True)
missing_driver_rating_fixed = initial_missing_driver_rating - df['driver_rating'].isnull().sum()

print(f"Mean Driver Rating Used to fill missing values: {mean_driver_rating:.2f}")
print(f"Number of missing 'driver_rating' values fixed: {missing_driver_rating_fixed}")

# Initialize MinMaxScaler
scaler = MinMaxScaler()

# Apply Min-Max normalization to 'driver_rating'
df['driver_rating_normalized'] = scaler.fit_transform(df[['driver_rating']])

print("\n--- Driver Rating Before and After Normalization (First 5 Rows) ---")
display(df[['driver_rating', 'driver_rating_normalized']].head())
print("\nNormalized Driver Rating Statistics:\n", df['driver_rating_normalized'].describe())
print("\n")
```

```
Mean Driver Rating Used to fill missing values: 4.19
Number of missing 'driver_rating' values fixed: 1
```

6. Generate Summary of Cleaning Operations

This section provides a consolidated summary of all the data cleaning and transformation steps performed.

```
print("--- Cleaning Summary ---")
print(f"Missing 'fare_amount' values fixed: {missing_fare_fixed}")
print(f"Missing 'driver_rating' values fixed: {missing_driver_rating_fixed}")
print(f"Duplicate ride entries removed: {duplicates_removed_count}")
print(f"Columns transformed: pickup_location (lowercase, stripped), drop_location (lowercase, stripped), driver_rating (normalized)\n")
```

Output:

```
df['driver_rating'].fillna(mean_driver_rating, inplace=True)
```

	driver_rating	driver_rating_normalized
0	4.5	0.714286
1	3.8	0.214286
2	4.9	1.000000
3	4.2	0.500000
4	3.5	0.000000

```
Normalized Driver Rating Statistics:
 count    19.000000
mean      0.492063
std       0.313065
min       0.000000
25%       0.250000
50%       0.428571
75%       0.750000
max       1.000000
Name: driver_rating_normalized, dtype: float64
```

```
--- Original Dataset (First 5 Rows for comparison) ---
```

	ride_id	pickup_location	drop_location	fare_amount	driver_rating	ride_timestamp
0	1	Downtown	Uptown	25.5	4.5	2023-01-01 10:00:00
1	2	Airport	Downtown	45.0	3.8	2023-01-01 11:30:00
2	3	Suburbs	Airport	NaN	4.9	2023-01-01 12:00:00
3	4	Downtown	University	20.0	4.2	2023-01-01 13:15:00
4	5	Airport	Suburbs	40.0	3.5	2023-01-01 14:00:00

```
--- Final Cleaned Dataset (First 5 Rows) ---
```

	ride_id	pickup_location	drop_location	fare_amount	driver_rating	ride_timestamp	driver_rating_normalized
0	1	downtown	uptown	25.5	4.5	2023-01-01 10:00:00	0.714286
1	2	airport	downtown	45.0	3.8	2023-01-01 11:30:00	0.214286
2	3	suburbs	airport	25.5	4.9	2023-01-01 12:00:00	1.000000
3	4	downtown	university	20.0	4.2	2023-01-01 13:15:00	0.500000
4	5	airport	suburbs	40.0	3.5	2023-01-01 14:00:00	0.000000

```
Cleaned dataset shape: (19, 7)

--- Final Missing Values Check (Should be all zeros) ---
ride_id                    0
pickup_location            0
drop_location              0
fare_amount                0
driver_rating              0
ride_timestamp             0
driver_rating_normalized   0
dtype: int64
```

Explanation: The program begins by loading the ride-sharing dataset with pandas and displaying the original data to allow comparison after processing. It

then standardizes the location columns by removing extra spaces and converting all text to lowercase, ensuring that identical locations are not treated differently due to formatting variations.

Missing values in the fare_amount column are filled using the median fare to avoid the impact of extreme values on the data. Duplicate records based on ride_id are eliminated to keep each ride entry unique and accurate. The driver_rating column is scaled using Min-Max normalization to bring all values into a uniform range. Finally, a summary is produced highlighting the number of corrections made, and the cleaned dataset is saved for further analysis.